

Cache com Redis em API Express

Objetivos da Aula

- Implementar cache com Redis
- Usar query params como chave de cache
- Melhorar performance de APIs

Por que usar cache?

- Reduz tempo de resposta
- Diminui carga no servidor
- Melhora experiência do usuário
- Ideal para dados acessados frequentemente

Configuração Inicial

```
const express = require("express");  
const redis = require("redis");  
  
const app = express();  
const redisClient = redis.createClient();
```

Estrutura de Cache

1. Recebe query param
2. Verifica no Redis
3. Se existir: retorna do cache
4. Se não existir:
 - Busca dados
 - Armazena no Redis
 - Retorna resultado

Implementação

```
app.get("/search", async (req, res) => {  
  const { query } = req.query;  
  
  redisClient.get(query, async (err, data) => {  
    if (data) return res.json(JSON.parse(data));  
  
    const result = await buscarDados(query);  
    redisClient.setex(query, 3600, JSON.stringify(result));  
    res.json(result);  
  });  
});
```

Testando a API

```
# Primeira chamada (armazena no cache)
curl "http://localhost:3000/search?query=produtos"

# Segunda chamada (pega do cache)
curl "http://localhost:3000/search?query=produtos"
```

Boas Práticas

- Definir TTL (Time-To-Live) adequado
- Invalidar cache quando dados mudam
- Monitorar uso de memória do Redis
- Considerar cache distribuído para alta escala

Passo a passo detalhado

1 Preparar o ambiente

- Node.js instalado (≥ 14)
- Redis em execução (`redis-server`)
- `mkdir cache-redis && cd cache-redis && npm init -y`

2 Instalar dependências

```
npm install express redis
```

3 Configurar o cliente Redis

```
const redis = require('redis');
const client = redis.createClient();
client.on('connect', () => console.log('✅ Conectado ao Redis'));
client.on('error', err => console.error('❌ Erro Redis:', err));
```

4 Criar servidor Express

```
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;
app.use(express.json());
```

5 Definir rota de cache

```
app.get('/search', async (req, res) => {  
  const { query } = req.query;  
  if (!query) return res.status(400).json({ error: 'Parâmetro "query" obrigatório' });  
  
  client.get(query, async (err, data) => {  
    if (data) return res.json(JSON.parse(data));  
  
    const result = await buscarDados(query);  
    client.setex(query, 3600, JSON.stringify(result));  
    res.json(result);  
  });  
});
```

6 Função de busca (exemplo)

```
async function buscarDados(query) {  
  return { query, resultado: `Dados para "${query}"` };  
}
```

7 Testar a API

```
curl "http://localhost:3000/search?query=produtos" # grava no cache  
curl "http://localhost:3000/search?query=produtos" # lê do cache
```

8 Boas práticas avançadas

- Definir TTL adequado (ex.: 3600 s)
- Invalidar cache ao atualizar dados (`client.del(key)`)
- Sanitizar chaves (usar `encodeURIComponent`)
- Monitorar memória (`INFO memory`)
- Tratar erros de conexão Redis

Exercício Prático

1. Implemente uma rota `/users` com cache
2. Use ID do usuário como chave
3. Defina TTL de 30 minutos
4. Teste com e sem cache

Referências

- Documentação Redis: <https://redis.io/documentation>
- Node Redis: <https://github.com/redis/node-redis>
- Express.js: <https://expressjs.com/>